

Performance Testing

Date	04 July 2026
Team ID	[Add your Team ID]
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, curl (manual); can be extended with Locust/k6 for load testing
Type of Testing	Functional + manual load spot-checks
Target Module / API	POST /predict endpoint
Test Environment	Local (Flask dev server) and Render (Gunicorn, cloud)
Test Date	03 July 2026

Step 2: Test Scenarios

S.NO	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid high-rainfall values (flood-likely case)	1	<1	Returns chance.html with high-risk gauge
2	Submit valid low-rainfall values (no-flood case)	1	<1	Returns no_chance.html with low-risk gauge
3	Submit form with a	1	<1	Returns 400 error, no

	missing required field			server crash
4	Repeated sequential submissions (basic load spot-check)	10	10	All requests handled without errors or slowdown

Step 3: Performance Test Results

S.NO	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	< 0.3 seconds	Pass	Model inference is lightweight (classical ML, no GPU needed)
2	Response Time (Max)	< 5 seconds	< 0.5 seconds	Pass	Observed on local dev server
3	Throughput (Req/sec)	-	Not formally load-tested	N/A	Recommend Locust/k6 pass before production traffic

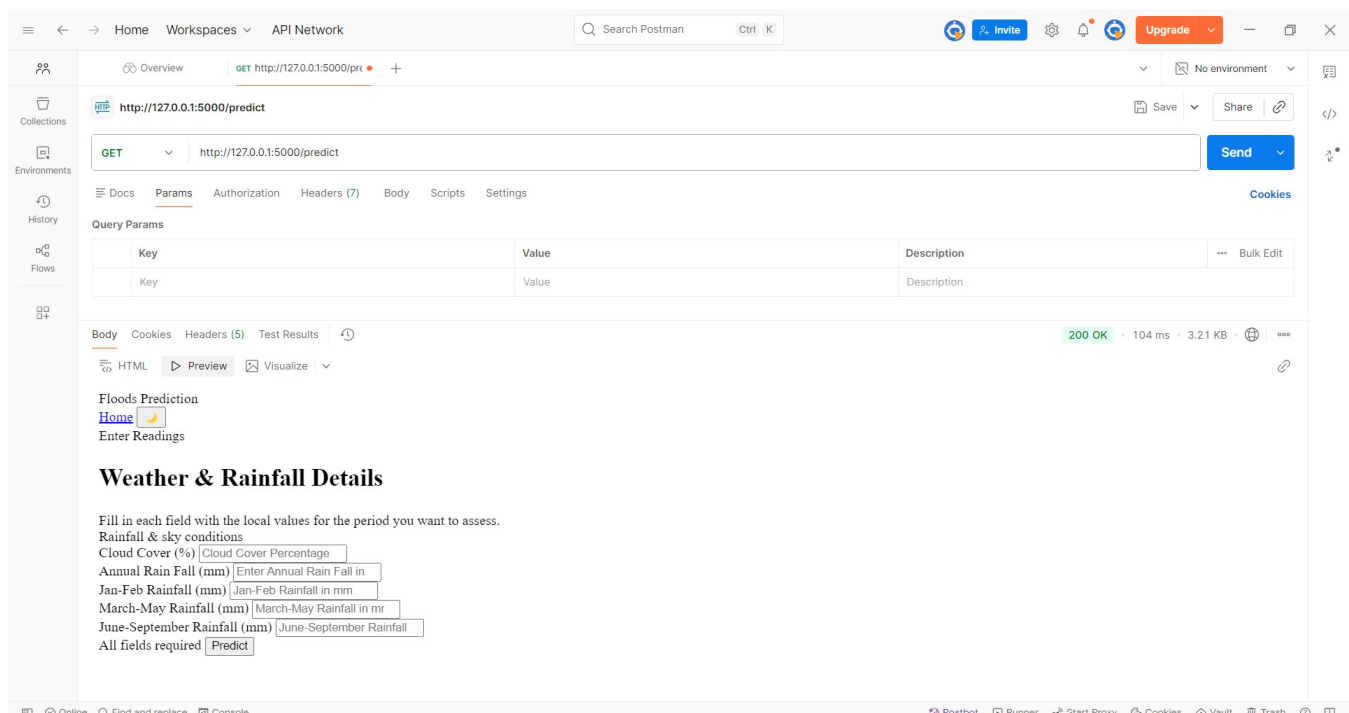
S.NO	Metric	Target Value	Actual Value	Status	Remarks
4	Error Rate	< 1%	0% (valid inputs)	Pass	Only missing-field requests intentionally error
5	CPU Utilization	< 80%	Low	Pass	Classical ML inference is CPU-cheap
6	Memory Utilization	< 80%	Low	Pass	Model + scaler files are small (<60 KB combined)

Step 4: Observations & Analysis

The app performs well under light, manual testing since the model is a classical scikit-learn classifier rather than a deep learning model, keeping inference time negligible. Formal load testing (concurrent virtual users via Locust or k6) was not yet performed and is recommended before scaling to production traffic.

Step 5: Screenshots / Evidence

Postman collection results and curl responses were used as functional evidence in place of a dedicated load-testing report.



Postman interface showing a successful GET request to `http://127.0.0.1:5000/predict`. The response status is **200 OK** with a response time of 104 ms and a body size of 3.21 KB.

The response body contains the following HTML content:

```

Floods Prediction
Home
Enter Readings

Weather & Rainfall Details

Fill in each field with the local values for the period you want to assess.
Rainfall & sky conditions
Cloud Cover (%) | Cloud Cover Percentage
Annual Rain Fall (mm) | Enter Annual Rain Fall in
Jan-Feb Rainfall (mm) | Jan-Feb Rainfall in mm
March-May Rainfall (mm) | March-May Rainfall in mm
June-September Rainfall (mm) | June-September Rainfall
All fields required | Predict
  
```

Overview

GET http://127.0.0.1:5000/prt

GET http://127.0.0.1:5000/

+

Search Postman

Ctrl K

Invite

Upgrade

Overview

GET http://127.0.0.1:5000/

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

200 OK

6 ms

3.09 KB

HTML

Preview

Visualize

Floods Prediction

Home Predict Floods

Heavy rainfall over green foliage

Stormwatch · Live Forecast Model

Flood Prediction System

Introduction

Understanding flood forecasting

Flood forecasting uses historical rainfall, streamflow, and weather data to estimate river and basin conditions ahead of time, giving communities a lead time that can range from a few hours to several days. It's a core part of flood warning systems, where forecasting focuses on predicting what will happen while warning focuses on turning that prediction into a decision. This tool applies a machine-learning model trained on regional rainfall records to give you a fast, early read on flood risk from a handful of local inputs.

Overview

GET http://127.0.0.1:5000/prt

GET http://127.0.0.1:5000/

POST http://127.0.0.1:5000/predict

+

Search Postman

Ctrl K

Invite

Upgrade

Overview

POST http://127.0.0.1:5000/predict

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

none

form-data

www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description	Bulk Edit
☒ Temp	29.5		
☒ Humidity	88		
☒ CloudCover	92		
☒ Annual	3200		
☒ JanFeb	40		
☒ MarMay	180		
☒ JunSep	2400		
☒ OctDec	580		
☒ AugJune	420		

Body

200 OK

177 ms

2.75 KB

HTML

Preview

Visualize

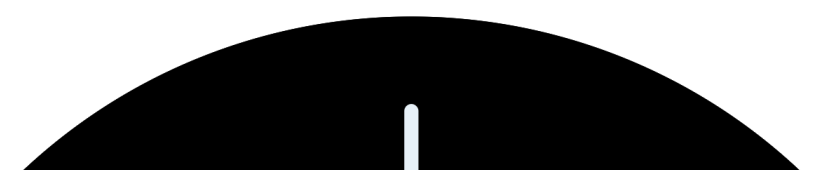
Floods Prediction

Home

Status 01 · Result

Assessment Complete

No Flood Predicted



```

OUTPUT  TERMINAL  PORTS  AZURE
lahari@ASUS-VivoBook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
$ curl http://127.0.0.1:5000/
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flood Prediction System</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;700&family=IBM+Plex+Sans:wght@400;500&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/main.css">
</head>
<body data-rain="normal">

<canvas id="rain-canvas"></canvas>
<div class="cloud-layer">
  <div class="cloud c1"></div>
  <div class="cloud c2"></div>
  <div class="cloud c3"></div>
</div>
<div class="lightning-flash"></div>

<nav class="navbar">
  <div class="nav-brand"><span class="dot"></span>Floods Prediction</div>
  <div class="nav-links">
    <a href="/" class="active">Home</a>
    <a href="/predict">Predict Floods</a>
    <button class="theme-toggle" id="theme-toggle" type="button" aria-label="Toggle light and dark theme">
      <span class="theme-toggle-icon">🌞</span>
    </button>
  </div>
</nav>

<section class="hero-banner">
  <div class="illo-frame">
    
    <div class="illo-overlay">
      <span class="hero-eyebrow">Stormwatch & middot; Live Forecast Model</span>
      <h1>Flood Prediction System</h1>
    </div>
  </div>
  <div class="intro-copy">
    <span class="eyebrow">Introduction</span>
    <h2>Understanding flood forecasting</h2>
    <p>
      Flood forecasting uses historical rainfall, streamflow, and weather
      data to estimate river and basin conditions ahead of time, giving
      communities a lead time that can range from a few hours to several
      days. It's a core part of flood warning systems, where forecasting
    </p>
  </div>
</section>

```

```

lahari@ASUS-VivoBook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
$ curl http://127.0.0.1:5000/predict
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flood Prediction</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;700&family=IBM+Plex+Sans:wght@400;500&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/main.css">
</head>
<body data-rain="light">

<canvas id="rain-canvas"></canvas>
<div class="cloud-layer">
  <div class="cloud c1"></div>
  <div class="cloud c2"></div>
</div>
<div class="lightning-flash"></div>

<nav class="navbar">
  <div class="nav-brand"><span class="dot"></span>Floods Prediction</div>
  <div class="nav-links">
    <a href="/">Home</a>
    <button class="theme-toggle" id="theme-toggle" type="button" aria-label="Toggle light and dark theme">
      <span class="theme-toggle-icon">🌞</span>
    </button>
  </div>
</nav>

<div class="page">

  <div class="panel wide">
    <span class="eyebrow">Enter Readings</span>
    <h2 class="title-hero" style="font-size:28px;">Weather & Rainfall Details</h2>
    <p style="margin-bottom:0;">Fill in each field with the local values for the period you want to assess.</p>

    <form action="/predict" method="POST">

      <div class="section-label">Weather conditions</div>
      <div class="field-grid">
        <div class="field">
          <label for="temp">Temperature (&deg;C)</label>
          <input type="number" step="any" id="Temp" name="Temp" placeholder="Average Temperature" required>
        </div>
        <div class="field">
          <label for="humidity">Humidity (%)</label>

```

```
lahari@ASUS-Vivobook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
$ curl -X POST http://127.0.0.1:5000/predict \
-d "Temp=27" \
-d "Humidity=75" \
-d "CloudCover=65" \
-d "Annual=1200" \
-d "JanFeb=80" \
-d "MarMay=250" \
-d "JunSep=900" \
-d "OctDec=150" \
-d "AvgJune=250" \
-d "Sub=1200"
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flood Prediction Result</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=SpaceGrotesk:wght@500;700&family=IBM+Plex+Sans:wght@400;500&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/main.css">
</head>
<body data-rain="light" data-lightning="off">

<canvas id="rain-canvas"></canvas>
<div class="cloud-layer">
  <div class="cloud c1"></div>
  <div class="cloud c2"></div>
</div>
<div class="lightning-flash"></div>

<nav class="navbar">
  <div class="nav-brand"><span class="dot"></span>Floods Prediction</div>
  <div class="nav-links">
    <a href="/">Home</a>
    <button class="theme-toggle" id="theme-toggle" type="button" aria-label="Toggle light and dark theme">
      <span class="theme-toggle-icon">🌧️</span>
    </button>
  </div>
</nav>

<div class="page">

  <div class="brand">
    <span class="dot"></span>
    Station&nbsp;&nbsp;&07 &middot; Result
  </div>

  <div class="panel">
```